ACCESO A DATOS - 2.º DAM

UP 1 – Ficheros - RandomAccessFile

Acceso aleatorio a ficheros en Java

0. try-with-resources: trabajar con recursos de forma segura

Cuando un programa trabaja con determinados elementos externos, como un fichero, necesita abrirlos o utilizarlos temporalmente.

A esos elementos Java los considera **recursos**.

En este tema, por ejemplo, cuando abrimos un fichero con `RandomAccessFile`, ese fichero es un recurso que nuestro programa está utilizando.

```
RandomAccessFile raf = new RandomAccessFile("datos.dat", "rw");
```

Mientras el recurso está abierto, podemos leer o escribir en el fichero.

El problema es que, cuando terminamos de utilizarlo, debemos **cerrarlo** para liberar correctamente los recursos del sistema.

Podríamos hacerlo manualmente:

```
RandomAccessFile raf = new RandomAccessFile("datos.dat", "rw");
// Trabajamos con el fichero
raf.close();
```

Pero esta forma tiene un inconveniente: si se produce un error antes de llegar a `close()`, el fichero podría no cerrarse correctamente.

Por eso Java ofrece una estructura llamada `try-with-resources`.

```
try (RandomAccessFile raf = new RandomAccessFile("datos.dat", "rw")) {
    // trabajar con el fichero
}
```

En este caso:

- `RandomAccessFile raf` es el recurso.
- El recurso se abre al entrar en el `try`.
- Dentro del bloque podemos utilizarlo normalmente.
- Cuando termina el bloque, Java lo cierra automáticamente.

No necesitamos escribir:

```
raf.close();
```

¿Qué ventaja tiene?

La principal ventaja es que nos aseguramos de que el recurso se cierre correctamente, incluso aunque durante la ejecución se produzca un error.

> IDEA CLAVE
> Un recurso es un elemento que nuestro programa abre o utiliza temporalmente y que después debe cerrarse. Con `try-with-resources`, Java se encarga automáticamente de ese cierre.

1. Dos formas de acceder a un fichero

Cuando guardamos información en un fichero, los datos quedan almacenados en el disco. Después, nuestro programa puede volver a abrir ese fichero para leerlos o modificarlos. La forma de llegar hasta un dato concreto puede ser distinta según el tipo de acceso que utilicemos.

1.1. Acceso secuencial

En un acceso **secuencial** los datos se recorren siguiendo el orden en el que aparecen en el fichero. Podemos imaginarlo como una fila: para llegar a un elemento lejano, vamos pasando por los anteriores.

> **Ejemplo**
> Si tenemos 10.000 registros y queremos localizar el registro 7.500, con un recorrido secuencial tendríamos que ir leyendo los registros anteriores hasta llegar a él.

1.2. Acceso aleatorio

En un acceso **aleatorio** podemos desplazarnos hasta una posición concreta del fichero sin tener que recorrer necesariamente todo lo anterior. Para hacerlo necesitamos saber en qué posición se encuentra el dato que buscamos.

> *Si conocemos exactamente dónde empieza el registro 7.500, ¿tiene sentido leer los 7.499 registros anteriores?*
>
> No. Si podemos calcular su posición, podemos desplazarnos directamente hasta ella.

1.3. ¿Qué papel tiene `RandomAccessFile`?

`RandomAccessFile` es una clase de Java que permite abrir un fichero para lectura, para escritura o para ambas operaciones y trabajar en posiciones concretas de su contenido.

Estas posiciones se expresan en bytes.

2. ¿Qué significa una «posición» dentro del fichero?

Para entender `RandomAccessFile` necesitamos recordar qué hay realmente dentro de un fichero: **bytes**. Un byte es una unidad de información. El contenido del fichero puede imaginarse como una secuencia de bytes numerados desde 0.

0	1	2	3	4	5	6	7
byte	byte	byte	byte	byte	byte	byte	byte

La posición 0 es el primer byte del fichero, la posición 1 es el segundo byte, etc. Cuando digamos «ir a la posición 20», significará «colocar la siguiente operación a partir del byte 20».

2.1. El puntero del fichero

`RandomAccessFile` **mantiene un puntero interno.** Ese puntero indica en qué posición del fichero se realizará la próxima lectura o escritura.

Ejemplo: Imagina un cursor
Es parecido al cursor de un editor de texto: antes de escribir, el cursor indica dónde aparecerá el siguiente carácter. Aquí el puntero indica desde qué byte se realizará la siguiente operación.

Al abrir un `RandomAccessFile`, el puntero comienza en la posición 0. Es decir, al principio del fichero.

3. Abrir un fichero con RandomAccessFile

Para utilizar la clase debemos crear un objeto `RandomAccessFile`. El constructor recibe dos datos: el fichero que queremos abrir y el modo de acceso.

```
RandomAccessFile raf = new RandomAccessFile("datos.dat", "rw");
```

En este ejemplo:

- "datos.dat" es el nombre del fichero.
- "rw" indica que queremos abrirlo para leer y para escribir.
- raf es la variable que utilizaremos para trabajar con ese fichero.

3.1. Modos de apertura

Modo	Qué permite	Qué ocurre
`r`	Sólo lectura	El fichero debe existir. Si no existe, se produce un error.
`rw`	Lectura y escritura	Si no existe, Java puede crearlo.
`rwd`	Lectura y escritura	Fuerza la sincronización de los cambios de datos con el dispositivo.
`rws`	Lectura y escritura	Sincroniza datos y también metadatos; puede ser más costoso.

Para los ejemplos de clase usaremos normalmente el modo "rw", porque nos permite leer y escribir en el mismo fichero.

4. ¿Cómo se mueve el puntero?

Antes de moverlo manualmente, vamos a observar qué ocurre cuando escribimos datos. `RandomAccessFile` tiene métodos específicos para escribir distintos tipos de datos.

```
try (RandomAccessFile raf = new RandomAccessFile("datos.dat", "rw")) {

    System.out.println(raf.getFilePointer());

    raf.writeInt(25);
    System.out.println(raf.getFilePointer());

    raf.writeDouble(7.5);
    System.out.println(raf.getFilePointer());
}
```

El método `getFilePointer()` devuelve la posición actual del puntero.

Momento	Posición del puntero	Por qué
Nada más abrir	0	Estamos al principio.
Después de writeInt(25)	4	Un int ocupa 4 bytes.
Después de writeDouble(7.5)	12	El double ocupa 8 bytes más.

> **Muy importante**
> El puntero avanza automáticamente cuando leemos o escribimos. No avanza «un dato»: avanza tantos bytes como ocupe ese dato.

5. Desplazando el puntero con seek()

Hasta aquí Java ha ido moviendo el puntero automáticamente. Pero la característica que hace especial a `RandomAccessFile` es que nosotros también podemos decidir en qué posición queremos colocarlo.

Para ello se utiliza el método `seek(long posicion)`. El número que pasamos indica el byte al que queremos mover el puntero.

```
raf.seek(0);             // mover el puntero al inicio
raf.seek(20);            // mover el puntero al byte 20
raf.seek(raf.length());  // mover el puntero al final del fichero
```

El método `length()` devuelve el tamaño actual del fichero en bytes. Por eso `seek(raf.length())` coloca el puntero justo al final.

> *¿*seek(20) *lee el byte 20?*
>
> **Respuesta:** No. seek() solo mueve el puntero. Para leer o escribir tenemos que ejecutar después un método como readInt() o writeInt().

6. Leer o escribir desde una posición concreta

Una vez colocado el puntero, la siguiente operación comienza exactamente desde esa posición.

```
raf.seek(0);
int numero = raf.readInt();     // lee un int desde el byte 0

raf.seek(0);
raf.writeInt(99);               // escribe un int desde el byte 0
```

> **Atención: escribir no significa insertar**
>
> Si en esa posición ya había datos, los bytes nuevos sustituyen a los que había.
> `RandomAccessFile` no desplaza automáticamente el resto del fichero para dejar hueco.

7. Ejemplo muy sencillo: modificar el segundo entero

Vamos a guardar tres enteros. Como cada int ocupa 4 bytes, podemos calcular fácilmente dónde comienza cada uno.

```
try (RandomAccessFile raf = new RandomAccessFile("numeros.dat", "rw")) {
    raf.setLength(0);     // vaciamos el fichero para repetir el ejemplo desde cero

    raf.writeInt(10);     // bytes 0-3
    raf.writeInt(20);     // bytes 4-7
    raf.writeInt(30);     // bytes 8-11
}
```

Dato	Bytes del fichero que ocupa	Posición inicial
10	bytes **0, 1, 2 y 3**	0
20	bytes **4, 5, 6 y 7**	4
30	bytes **8, 9, 10 y 11**	8

`writeInt()` escribe 4 bytes porque un int en Java ocupa 4 bytes**.**

Igual que:

- writeDouble() → 8 bytes
- writeChar() → 2 bytes
- writeBoolean() → 1 byte

Si queremos cambiar solo el 20 por 99**,** no hace falta volver a escribir los tres números. El segundo entero empieza en el byte 4.

```
try (RandomAccessFile raf = new RandomAccessFile("numeros.dat", "rw")) {
    raf.seek(4);
    raf.writeInt(99);
}
```

Resultado → El fichero sigue conteniendo tres enteros, pero ahora son 10, 99 y 30.

8. Leer correctamente exige conocer la estructura del fichero

El fichero no guarda etiquetas que digan «aquí hay un int» o «aquí empieza un double». Solo contiene bytes. Es nuestro programa quien debe saber qué se escribió y en qué orden.

Escritura

```
raf.writeInt(25);
raf.writeDouble(7.5);
```

La lectura correcta debe respetar esa misma estructura:

Lectura

```
int edad = raf.readInt();
double nota = raf.readDouble();
```

> **¿Qué ocurre si usamos el método equivocado?**
> Java leerá los bytes según el tipo que le indiquemos. El resultado será incorrecto o la lectura puede fallar. Por eso hay que conocer el formato del fichero.

8.1. ¿Y las cadenas de texto?

Las cadenas de texto requieren algo más de cuidado porque no siempre ocupan el mismo número de bytes.

Con `RandomAccessFile`, podemos guardar una cadena utilizando `writeUTF()`:

```
raf.writeUTF("Java");
```

Este método no guarda solo el texto. También almacena la información necesaria para saber **cuánto ocupa esa cadena** y poder recuperarla después correctamente.

Para leerla utilizamos el método correspondiente:

```
String texto = raf.readUTF();
```

Esto funciona sin problema si leemos los datos en el mismo orden en que se escribieron.

9. Registros de longitud fija

Un registro es un conjunto de datos que pertenecen a una misma entidad. Por ejemplo, un videojuego puede tener id, precio y stock.
Si todos los registros ocupan exactamente el mismo número de bytes, podemos calcular dónde empieza cualquiera de ellos sin leer los anteriores.

> **Fórmula básica**
> posición inicial = número de registro × tamaño del registro

> Si cada registro ocupa 16 bytes, ¿dónde empieza el registro 0? ¿Y el 1? ¿Y el 2?
> **Respuesta**: En los bytes 0, 16 y 32 respectivamente.

9.1. Importante: numeramos los registros desde 0

Cuando utilizamos la fórmula, solemos numerar los registros como 0, 1, 2, 3... Por tanto, el «tercer registro» para una persona corresponde al número de registro 2 en el cálculo.

10. Ejemplo aplicado a LudotecaIES

Hasta ahora hemos trabajado con ejemplos sencillos para entender cómo funciona el acceso aleatorio. Vamos a aplicar ahora esas ideas a nuestro proyecto **LudotecaIES**.

En el proyecto, un videojuego contiene varios datos, por ejemplo:

```
private int id;
private String titulo;
```

```
private String plataforma;
private double precio;
private int stock;
```

Sin embargo, para practicar con `RandomAccessFile` no vamos a guardar todavía el objeto completo. El motivo es que título y plataforma son objetos String y su longitud puede variar. Por ejemplo:

```
"Tetris"
```

no ocupa lo mismo que:

```
"Super Mario Bros"
```

Esto hace más difícil calcular directamente dónde comienza cada videojuego dentro del fichero. Por eso, en este primer ejemplo vamos a trabajar únicamente con los campos que tienen tamaño fijo:

Campo	Tipo	Tamaño
id	int	4 bytes
precio	double	8 bytes
stock	int	4 bytes

Cada registro ocupará: 4 + 8 + 4 = **16 bytes**

10.1. ¿Qué vamos a guardar?

Supongamos que en nuestro catálogo tenemos estos videojuegos:

```
Videojuego videojuego1 = new Videojuego(1, "Tetris", "PC", 19.99, 5);
Videojuego videojuego2 = new Videojuego(2, "Super Mario Bros", "Switch", 29.99, 3);
Videojuego videojuego3 = new Videojuego(3, "Pac-Man", "PC", 14.99, 8);
```

De cada videojuego guardaremos en el fichero únicamente: `id + precio + stock`

Por ejemplo, de Tetris se almacenará:

```
1 | 19.99 | 5
```

De Super Mario Bros:

```
2 | 29.99 | 3
```

Y de Pac-Man:

```
3 | 14.99 | 8
```

Como siempre escribimos un int, un double y otro int, todos los registros ocupan exactamente 16 bytes.

10.2. Guardar los datos en el fichero

Vamos a guardar en el fichero los datos de los tres videojuegos. De cada uno almacenaremos siempre los campos en el mismo orden:

```
id → precio → stock
```

El código sería:

```
try (RandomAccessFile raf = new RandomAccessFile("datosVideojuegos.dat", "rw")) {

    raf.setLength(0);

    raf.writeInt(videojuego1.getId());
    raf.writeDouble(videojuego1.getPrecio());
    raf.writeInt(videojuego1.getStock());

    raf.writeInt(videojuego2.getId());
    raf.writeDouble(videojuego2.getPrecio());
    raf.writeInt(videojuego2.getStock());

    raf.writeInt(videojuego3.getId());
    raf.writeDouble(videojuego3.getPrecio());
    raf.writeInt(videojuego3.getStock());
}
```

La instrucción:

```
raf.setLength(0);
```

deja el fichero vacío antes de empezar a escribir. La utilizamos en este ejemplo para asegurarnos de que el fichero comienza desde cero cada vez que ejecutamos el programa. Así evitamos que queden datos de ejecuciones anteriores mezclados con los nuevos.

Importante: `setLength(0)` borra todo el contenido del fichero, por lo que solo debemos utilizarlo cuando realmente queramos empezar de nuevo.

10.3. ¿Cómo queda organizado el fichero?

Cada videojuego ocupa 16 bytes. Por tanto:

```
Registro 0
Tetris
bytes 0 - 15

Registro 1
Super Mario Bros.
bytes 16 - 31

Registro 2
Pac-Man
bytes 32 - 47
```

Podemos representarlo así:



Esto nos permite calcular fácilmente dónde empieza cada registro.

La fórmula es:

```
posición = número de registro × tamaño del registro
```

En nuestro caso:

```
posición = número de registro × 16
```

10.4. Acceder directamente a un videojuego

Queremos acceder, por ejemplo, a Super Mario Bros**.**
Aunque para nosotros sea el segundo videojuego, dentro del fichero los registros los numeraremos empezando desde 0:

```
Tetris              → registro 0
Super Mario Bros    → registro 1
Pac-Man             → registro 2
```

Por tanto:

```
int numeroRegistro = 1;
int TAM_REGISTRO = 16;
long posicion = numeroRegistro * TAM_REGISTRO;
```

El cálculo sería:

```
1 × 16 = 16
```

Super Mario Bros empieza en el byte 16. Podemos mover directamente el puntero hasta esa posición:

```
raf.seek(16);
```

O, mejor todavía, utilizando el cálculo:

```
raf.seek(numeroRegistro * TAM_REGISTRO);
```

A partir de esa posición podemos leer los tres campos:

```
int id = raf.readInt();
double precio = raf.readDouble();
int stock = raf.readInt();
```

Por ejemplo:

```
try (RandomAccessFile raf = new RandomAccessFile("datosVideojuegos.dat", "r")) {

    int numeroRegistro = 1;
    int TAM_REGISTRO = 16;

    raf.seek(numeroRegistro * TAM_REGISTRO);

    int id = raf.readInt();
    double precio = raf.readDouble();
    int stock = raf.readInt();

    System.out.println("Id: " + id);
    System.out.println("Precio: " + precio);
    System.out.println("Stock: " + stock);
}
```

10.5. Acceder directamente al stock

Ahora queremos modificar solamente el stock de Super Mario Bros y dentro de cada registro los datos se guardan en este orden:

```
id        → 4 bytes
precio    → 8 bytes
stock     → 4 bytes
```

Por tanto, el stock empieza después de:

```
4 + 8 = 12 bytes
```

Es decir:

```
inicio del registro + 12
```

Como Super Mario Bros está en el registro 1:

```
inicio del registro = 1 × 16 = 16
```

Y el stock empieza en:

```
16 + 12 = 28
```

Así que el stock de Super Mario Bros comienza en el byte 28.

10.6. Modificar solamente el stock

Supongamos que queremos cambiar el stock de Super Mario Bros de 3 unidades a 10.

```
try (RandomAccessFile raf = new RandomAccessFile("datosVideojuegos.dat", "rw")) {
    int numeroRegistro = 1;
    int TAM_REGISTRO = 16;

    long inicioRegistro = numeroRegistro * TAM_REGISTRO;
```

```
    long posicionStock = inicioRegistro + 12;

    raf.seek(posicionStock);
    raf.writeInt(10);
}
```

Paso a paso:

```
long inicioRegistro = numeroRegistro * TAM_REGISTRO;
```

Calcula dónde empieza el registro de Super Mario Bros:

```
1 × 16 = 16
```

Después:

```
long posicionStock = inicioRegistro + 12;
```

Calcula dónde empieza su stock:

```
16 + 12 = 28
```

Finalmente:

```
raf.seek(posicionStock);
```

mueve el puntero hasta el byte 28. Y:

```
raf.writeInt(10);
```

sustituye el valor anterior del stock por 10.

10.7. ¿Qué hemos modificado realmente?

Antes:

```
Super Mario Bros

id      → 2
precio  → 29.99
stock   → 3

Después:
Super Mario Bros

id      → 2
precio  → 29.99
stock   → 10
```

El id y el precio no se han modificado.

Tampoco hemos tenido que:

- leer Tetris;
- volver a escribir Pac-Man;
- recorrer todo el fichero;
- reescribir todos los videojuegos.

Hemos calculado exactamente dónde estaba el dato que queríamos modificar y hemos ido directamente hasta esa posición.

Idea clave: conocemos el tamaño de cada registro y la posición que ocupa cada campo dentro de ese registro. Gracias a ello podemos calcular exactamente dónde está un dato y modificarlo directamente.

10.8. Fórmula general

Para localizar un registro:

```
posiciónRegistro = numeroRegistro × tamañoRegistro
```

Para localizar un campo dentro de ese registro:

```
posiciónCampo = posiciónRegistro + desplazamientoCampo
```

En nuestro ejemplo:

```
Tamaño del registro = 16 bytes
```

Y los desplazamientos son:

Campo	Posición dentro del registro
id	0
precio	4
stock	12

Por ejemplo, para modificar el precio de Pac-Man, que está en el registro 2:

```
posiciónRegistro = 2 × 16 = 32
```

El precio empieza 4 bytes después del inicio del registro:

```
posiciónPrecio = 32 + 4 = 36
```

Por tanto:

```
raf.seek(36);
```

nos colocaría exactamente dónde empieza el precio de Pac-Man.

10.9. ¿Por qué no hemos guardado título y plataforma?

Porque queremos que todos los registros tengan **longitud fija**.

Los campos:

```
String titulo;
String plataforma;
```

pueden contener textos de distinta longitud.

Por ejemplo:

```
"Tetris"
"Super Mario Bros"
```

no ocupan necesariamente el mismo número de bytes.

Si cada registro tuviera un tamaño diferente, ya no podríamos localizarlo simplemente mediante:

```
numeroRegistro × tamañoRegistro
```

Por eso, en este primer ejemplo de acceso aleatorio utilizamos únicamente campos de tamaño fijo.

11. Modificar un campo concreto del registro

Ya hemos visto cómo localizar un videojuego completo dentro del fichero. Ahora vamos a ir un paso más allá: modificar únicamente uno de sus campos.

Queremos cambiar el stock de Super Mario Bros, que se encuentra en el registro 1.
Dentro de cada registro los datos están almacenados siempre en el mismo orden:

Campo	Tipo	Tamaño	Desplazamiento dentro del registro
id	int	4 bytes	0
precio	double	8 bytes	4
stock	int	4 bytes	12

El stock comienza, por tanto, 12 bytes después del inicio del registro.

Como Super Mario Bros está en el registro 1:

```
inicioRegistro = 1 × 16 = 16
```

Y su stock empieza en:

```
posicionStock = 16 + 12 = 28
```

Código:

```
try (RandomAccessFile raf = new RandomAccessFile("datosVideojuegos.dat", "rw")) {

    int numeroRegistro = 1;
    int TAM_REGISTRO = 16;

    long inicioRegistro = numeroRegistro * TAM_REGISTRO;

    long posicionStock = inicioRegistro + 12;

    raf.seek(posicionStock);
    raf.writeInt(10);
}
```

Antes:

```
Super Mario Bros
id      → 2
precio  → 29.99
stock   → 3
```

Después:

```
Super Mario Bros
id      → 2
precio  → 29.99
stock   → 10
```

Idea clave: no hemos modificado el registro completo. Hemos calculado la posición exacta del campo stock y hemos escrito únicamente ese dato.

12. Añadir un nuevo videojuego al final del fichero

Cuando abrimos un `RandomAccessFile`, el puntero se sitúa inicialmente en la posición 0.
Si el fichero ya contiene información y escribimos directamente, podríamos sobrescribir los primeros datos.

Para añadir un nuevo registro debemos colocar primero el puntero al final del fichero.
Podemos hacerlo con:

```
raf.seek(raf.length()); // devuelve el tamaño actual del fichero en bytes
```

Si tenemos tres videojuegos:

```
Tetris            → bytes 0–15
Super Mario Bros  → bytes 16–31
Pac-Man           → bytes 32–47
```

el fichero ocupa: 48 bytes, por tanto:

```
raf.seek(raf.length());
```

colocará el puntero en el byte 48, justo después del último registro.
Podemos añadir, por ejemplo, un cuarto videojuego:

```
try (RandomAccessFile raf =
        new RandomAccessFile("datosVideojuegos.dat", "rw")) {

    raf.seek(raf.length());

    raf.writeInt(4);
    raf.writeDouble(39.99);
    raf.writeInt(6);
}
```

El nuevo registro ocupará: registro 3 → bytes 48–63

¿Qué ocurriría si quitamos raf.seek(raf.length())?
Respuesta: *La escritura empezaría en la posición actual del puntero. Al abrir el fichero, normalmente sería 0, por lo que podríamos sobrescribir el registro de Tetris.*

13. Otros métodos útiles de `RandomAccessFile`

Además de `seek()`, utilizaremos otros métodos que nos permiten conocer o modificar el estado del fichero:

Método	Para qué sirve
`getFilePointer()`	Devuelve la posición actual del puntero.
`seek(long pos)`	Mueve el puntero a una posición concreta.
`length()`	Devuelve el tamaño del fichero en bytes.

`setLength(long n)`	Cambia la longitud del fichero.
`skipBytes(int n)`	Avanza un número de bytes desde la posición actual.
`readInt()`	Lee un valor int.
`readDouble()`	Lee un valor double.
`writeInt()`	Escribe un valor int.
`writeDouble()`	Escribe un valor double.

En nuestros ejemplos hemos utilizado:

```
raf.setLength(0);
```

para dejar el fichero vacío antes de volver a generar los datos. Y:

```
raf.length();
```

para saber dónde termina el fichero.

No necesitamos llamar manualmente a:

```
raf.close();
```

porque estamos utilizando `try-with-resources`.

14. Errores frecuentes

Al trabajar con `RandomAccessFile` conviene evitar algunos errores habituales.

Confundir número de registro con posición

```
raf.seek(2);
```

no significa “ir al registro 2”. Significa: ir al byte 2

Para ir al registro 2 de nuestro ejemplo debemos calcular: 2 × 16 = 32 y utilizar:

```
raf.seek(32);
```

Pensar que seek() lee información

`seek()` únicamente mueve el puntero.

```
raf.seek(16);
```

nos coloca al principio de Super Mario Bros, pero todavía no hemos leído nada.

Después tendríamos que hacer:

```
int id = raf.readInt();
```

Olvidar que leer o escribir mueve el puntero

Si hacemos:

```
raf.readInt();
```

el puntero avanza 4 bytes.

Si después hacemos:

```
raf.readDouble();
```

avanza otros 8.

Pensar que escribir inserta datos

Si escribimos en una posición que ya contiene información:

```
raf.seek(28);
```

```
raf.writeInt(10);
```

los nuevos bytes sustituyen a los que había.

No se crea espacio adicional.
No respetar la estructura del fichero
Si hemos escrito:

```
int + double + int
```

debemos leer esos datos respetando su tipo y su orden.

Usar registros de tamaño variable y aplicar la misma fórmula
La fórmula: **numeroRegistro × tamañoRegistro**
solo es sencilla porque todos nuestros registros ocupan 16 bytes.

> **Regla práctica**
> Antes de utilizar `seek()`, pregúntate siempre: ¿a qué byte quiero ir y cómo he calculado esa posición?

15. Ejemplo completo aplicado a LudotecaIES

En este ejemplo reunimos todo lo visto hasta ahora. Vamos a:

- crear tres objetos Videojuego;
- guardar en el fichero únicamente id, precio y stock;
- modificar directamente el stock de Super Mario Bros;
- volver al inicio de su registro;
- leer sus datos para comprobar que el cambio se ha realizado correctamente.

Como ya hemos visto, cada registro ocupa 16 bytes:

```
id      → 4 bytes
precio  → 8 bytes
stock   → 4 bytes
----------------
TOTAL   → 16 bytes
```

Por tanto, mantendremos este valor en una constante:

```
private static final int TAM_REGISTRO = 16;
```

El programa completo sería:

```
import java.io.IOException;
import java.io.RandomAccessFile;

public class EjemploRandomAccessFile {
    private static final int TAM_REGISTRO = 16;

    public static void main(String[] args) {
        Videojuego videojuego1 = new Videojuego(1, "Tetris", "PC", 19.99, 5);
        Videojuego videojuego2 = new Videojuego(2, "Super Mario Bros", "Switch", 29.99, 3);
```

```
        Videojuego videojuego3 = new Videojuego(3, "Pac-Man", "PC", 14.99, 8);

        try (RandomAccessFile raf = new RandomAccessFile("datosVideojuegos.dat", "rw")) {
            raf.setLength(0);  // Dejamos el fichero vacío

            // Guardamos los tres registros
            escribirRegistro(raf, videojuego1);
            escribirRegistro(raf, videojuego2);
            escribirRegistro(raf, videojuego3);

            // Super Mario Bros está en el registro 1
            int numeroRegistro = 1;

            // Calculamos dónde empieza su campo stock
            long posicionStock = numeroRegistro * TAM_REGISTRO + 12;

            // Nos situamos directamente en el stock
            raf.seek(posicionStock);

            // Cambiamos el stock de 3 a 10
            raf.writeInt(10);

            // Volvemos al inicio del registro de Super Mario Bros
            raf.seek(numeroRegistro * TAM_REGISTRO);

            // Leemos el registro completo
            int id = raf.readInt();
            double precio = raf.readDouble();
            int stock = raf.readInt();

            System.out.println("Id: " + id);
            System.out.println("Precio: " + precio);
            System.out.println("Stock: " + stock);

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static void escribirRegistro(
            RandomAccessFile raf,
            Videojuego videojuego) throws IOException {

        raf.writeInt(videojuego.getId());
        raf.writeDouble(videojuego.getPrecio());
        raf.writeInt(videojuego.getStock());
    }
}
```

15.1. ¿Por qué utilizamos escribirRegistro()?

Los tres videojuegos se almacenan siguiendo exactamente la misma estructura:

```
id + precio + stock
```

En lugar de repetir tres veces:

```
raf.writeInt(...);
raf.writeDouble(...);
raf.writeInt(...);
```

creamos un método auxiliar:

```
private static void escribirRegistro(RandomAccessFile raf, Videojuego videojuego) throws
IOException {
    raf.writeInt(videojuego.getId());
    raf.writeDouble(videojuego.getPrecio());
    raf.writeInt(videojuego.getStock());
}
```

El método recibe el `RandomAccessFile` y el videojuego que queremos guardar, y escribe siempre sus datos en el mismo orden.

Así podemos escribir cada registro con:

```
escribirRegistro(raf, videojuego1);
escribirRegistro(raf, videojuego2);
escribirRegistro(raf, videojuego3);
```

y todos mantienen la misma estructura y el mismo tamaño.

> **Idea Clave**
> hemos accedido directamente al campo que queríamos modificar y después hemos vuelto al comienzo del registro para comprobar el resultado. No ha sido necesario recorrer ni reescribir los demás registros.

16. Acceso aleatorio en ficheros de texto

Hasta ahora hemos utilizado `RandomAccessFile` principalmente con datos de **tamaño fijo**, como int y double. Esto nos permite calcular fácilmente la posición exacta de un dato dentro del fichero.

También podemos utilizar `RandomAccessFile` con ficheros que contienen texto, pero debemos tener un poco más de cuidado.

En un fichero de texto, las líneas pueden tener **longitudes diferentes**, por lo que no siempre podemos calcular su posición utilizando una fórmula como:

```
númeroRegistro × tamañoRegistro
```

En estos casos podemos aprovechar el **puntero del fichero** para recordar dónde empieza una determinada línea.

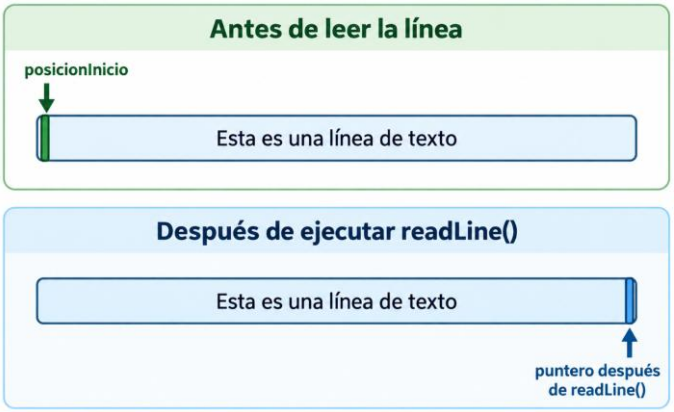
16.1. Guardar la posición antes de leer

Supongamos que tenemos un fichero de texto y queremos recorrerlo línea a línea.

Antes de leer cada línea podemos consultar dónde se encuentra el puntero:

```
long posicionInicio = raf.getFilePointer();
String linea = raf.readLine();
```

En este momento ocurren dos cosas:



`getFilePointer()` nos permite guardar **la posición donde empieza la línea**.

Después, `readLine()` lee la línea y hace avanzar el puntero.

Por tanto:

```
long posicionInicio = raf.getFilePointer();
String linea = raf.readLine();
```

nos permite conocer tanto:

- dónde comenzaba la línea;
- como su contenido.

16.2. Volver al comienzo de la línea

Si después de leerla necesitamos modificarla, podemos volver a la posición que habíamos guardado:

```
raf.seek(posicionInicio);
```

Por ejemplo:

```
long posicionInicio = raf.getFilePointer();

String linea = raf.readLine();

if (linea.contains("java")) {
    raf.seek(posicionInicio);

    // escribir la línea modificada
}
```

Aquí se ve otra utilidad de seek(): no necesitamos saber de antemano en qué byte empieza cada línea. **Guardamos su posición antes de leerla y podemos volver a ella después.**

16.3. Cuidado al sobrescribir texto

Hay una diferencia importante respecto al ejemplo de los enteros.

Si sustituimos un texto por otro que ocupa **más bytes**, podríamos sobrescribir parte de la información que viene después.

Por ejemplo, no sería seguro sustituir:

```
Java
```

por:

```
Programación Java
```

directamente en la misma posición.

El nuevo texto necesita más espacio.

Sin embargo, si hacemos una modificación que mantiene la misma longitud, el problema es mucho más sencillo.

Por ejemplo:

```
java
```

por:

```
JAVA
```

La palabra sigue teniendo cuatro caracteres.

> **Idea Clave**
> cuando trabajamos con texto de longitud variable, normalmente no podemos calcular tan fácilmente la posición de cada dato. Podemos utilizar `getFilePointer()` para guardar la posición antes de leer y `seek()` para regresar después a ella.

16.4. Ejemplo básico

```
try (RandomAccessFile raf = new RandomAccessFile("texto.txt", "rw")) {
    while (raf.getFilePointer() < raf.length()) {
        long posicionInicio = raf.getFilePointer();
        String linea = raf.readLine();
        System.out.println("La línea comienza en el byte " + posicionInicio + ": "+ linea);
    }
}
```

Aquí todavía **no modificamos nada**.

Simplemente observamos:

1. dónde empieza cada línea;
2. cómo `readLine()` hace avanzar el puntero;
3. cómo las diferentes líneas pueden comenzar en posiciones distintas.

Si antes de leer una línea el puntero estaba en el byte 35 y después de ejecutar:

String linea = raf.readLine();

está en el byte 58:

¿qué posición tendríamos que utilizar con seek() para volver al comienzo de esa línea?

raf.seek(35);

17. Resumen final

En este apartado hemos visto que `RandomAccessFile` permite trabajar con un fichero **sin tener que recorrerlo siempre desde el principio**.

Su funcionamiento se basa en un **puntero** que indica la posición actual dentro del fichero. Podemos consultar esa posición, movernos a otra y leer o escribir desde allí.

Concepto	Qué debes recordar
Acceso secuencial	Los datos se recorren siguiendo el orden en el que aparecen en el fichero.
Acceso aleatorio	Permite acceder directamente a una posición concreta del fichero.
Puntero	Indica el byte en el que se realizará la siguiente lectura o escritura.
`getFilePointer()`	Devuelve la posición actual del puntero.
`seek(pos)`	Mueve el puntero hasta la posición indicada.
`length()`	Devuelve el tamaño total del fichero en bytes.
`setLength(0)`	Deja el fichero vacío.
Lectura y escritura	Al leer o escribir, el puntero avanza según los bytes ocupados por el dato.
Sobrescritura	Escribir sobre una posición existente sustituye los bytes que había allí.
Registro de longitud fija	Todos los registros ocupan el mismo número de bytes, lo que permite calcular fácilmente su posición.
Posición de un registro	numeroRegistro * TAM_REGISTRO
Posición de un campo	inicioRegistro + desplazamientoCampo
Texto de longitud variable	No siempre podemos calcular directamente dónde empieza cada dato.
Posición de una línea	Podemos guardar su posición con getFilePointer() antes de leerla y volver después con seek()

Idea Clave

para utilizar bien RandomAccessFile necesitamos saber cómo está organizado el fichero. Si conocemos la posición de los datos, podemos acceder a ellos directamente sin recorrer todo el contenido.